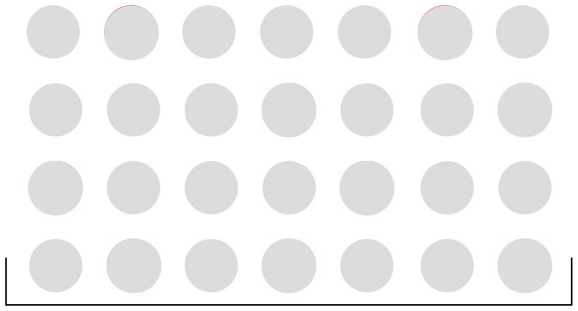
Experimentation & Evaluation

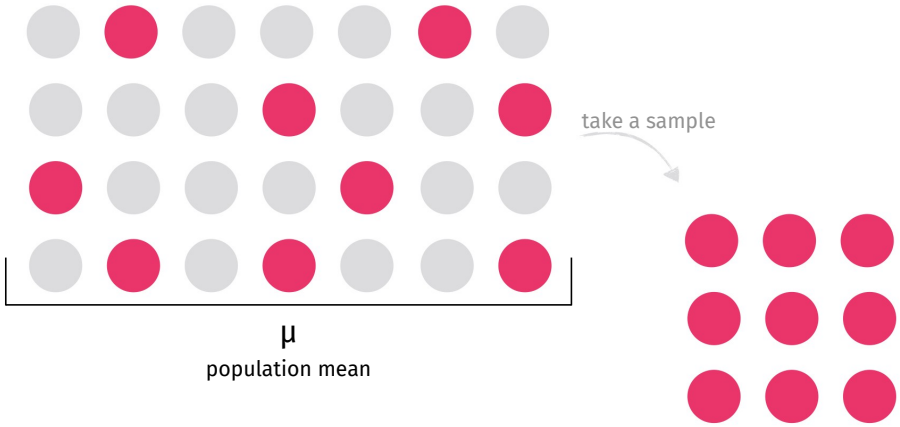
Gabriele Bavota
Giuseppe Crupi

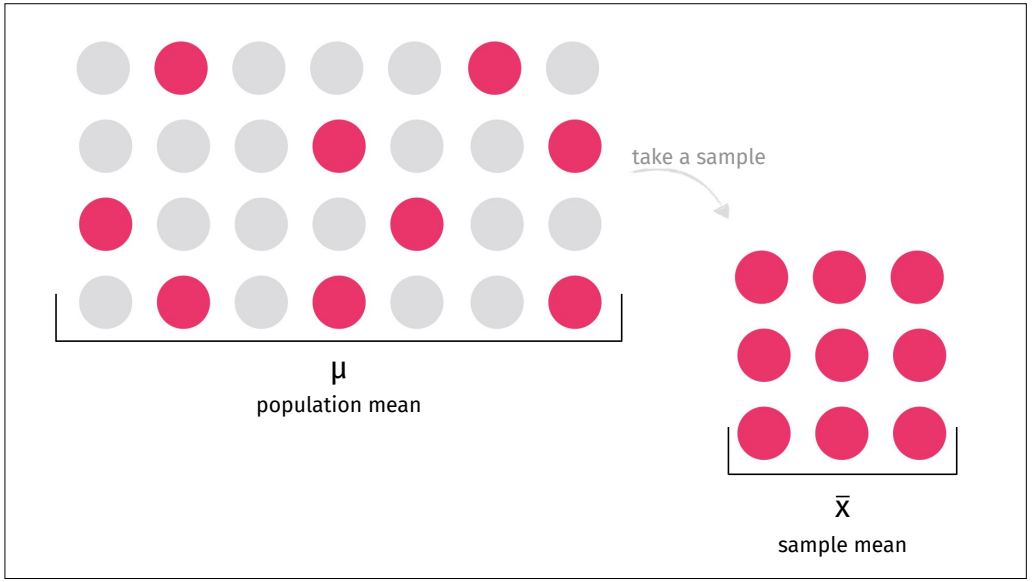


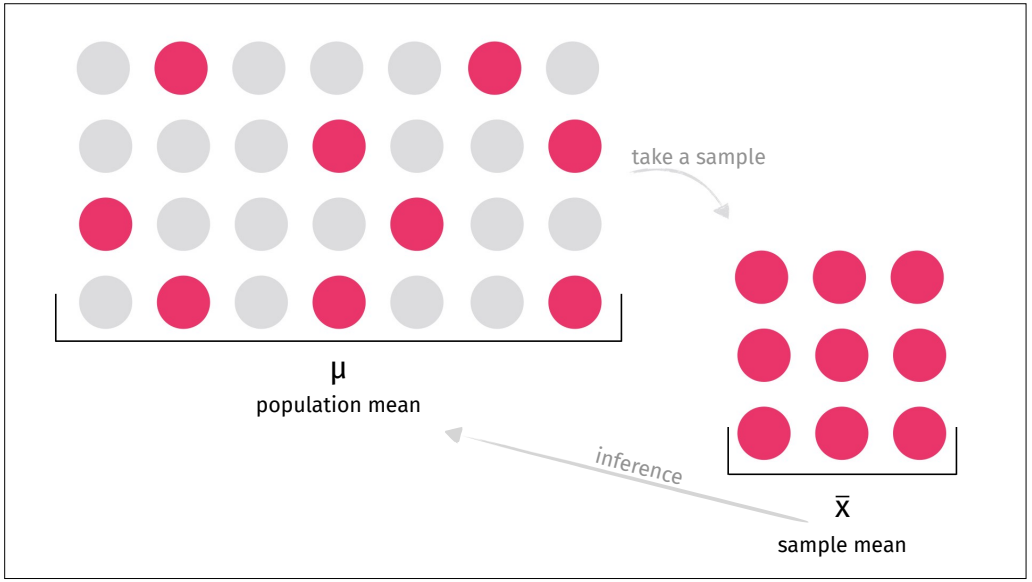
Lecture 05

Descriptive Statistics & Visualization









The sample must be representative of the population under study

Sampling is far from trivial

Sampling is connected to the generalizability of the results

Probability *vs* non-probability sample

Probability

Simple random sampling
Subjects are selected from a list of the population at random.

Stratified random sampling
The population is divided into a number of groups or strata with a known distribution between the groups. Random sampling is then applied within the strata.

Non-probability

Convenience sampling
The nearest and most convenient persons are selected as subjects.

Quota sampling
This type of sampling is used to get subjects from various elements of a population. Convenience sampling is normally used for each element.

Sampling Bias

Three (main) kinds of sampling bias

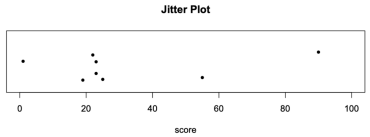
Self-selection bias
Online survey to investigate whether people use computers

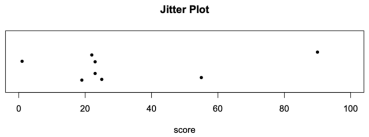
Undercoverage bias
Survey of Swiss uni students, only one USI student

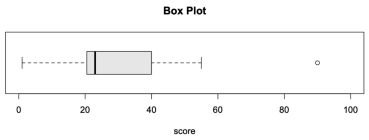
Survivorship bias
Survey about quality of this course, but only to students who didn't drop the course

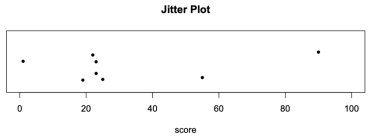
Visualize distribution

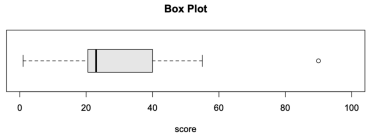
Visualizing a single distribution

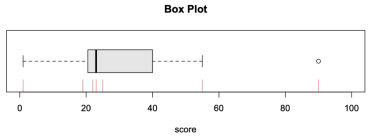


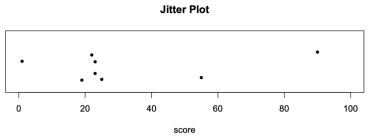


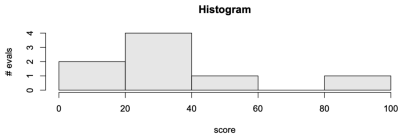


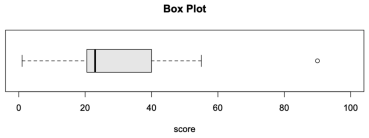

Jitter Plot
0
20
40
60
80
100
score

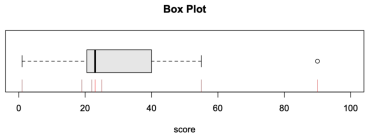

Box Plot
0
20
40
60
80
100
score

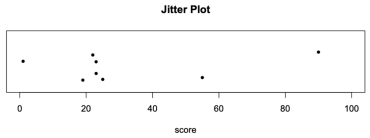

Box Plot
0
20
40
60
80
100
score

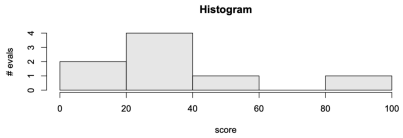

Jitter Plot
0
20
40
60
80
100
score

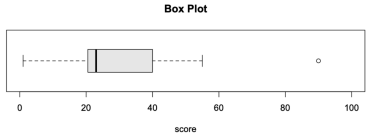

Histogram
evals
0
1
2
3
4
0
20
40
60
80
100
score

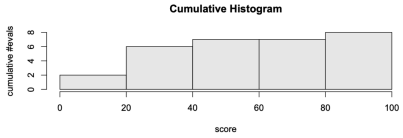

Box Plot
0
20
40
60
80
100
score

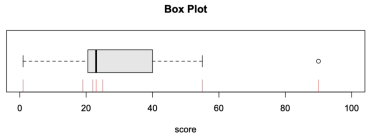

Box Plot
0
20
40
60
80
100
score

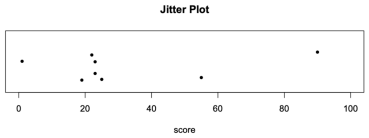

Jitter Plot
0
20
40
60
80
100
score

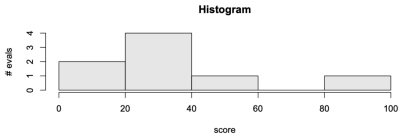

Histogram
evals
0
1
2
3
4
0
20
40
60
80
100
score

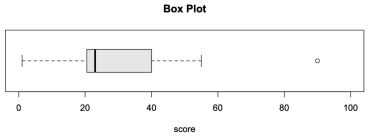

Box Plot
0
20
40
60
80
100
score

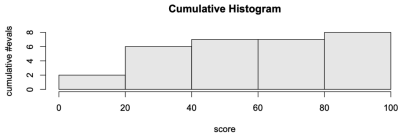

Cumulative Histogram
cumulative #evals
0
2
4
6
8
0
20
40
60
80
100
score

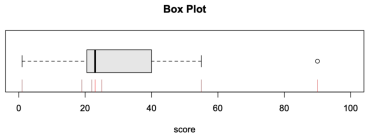

Box Plot
0
20
40
60
80
100
score

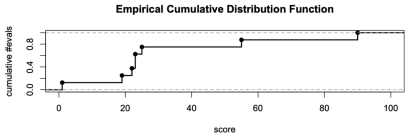

Jitter Plot
0
20
40
60
80
100
score


Histogram
evals
0
1
2
3
4
0
20
40
60
80
100
score

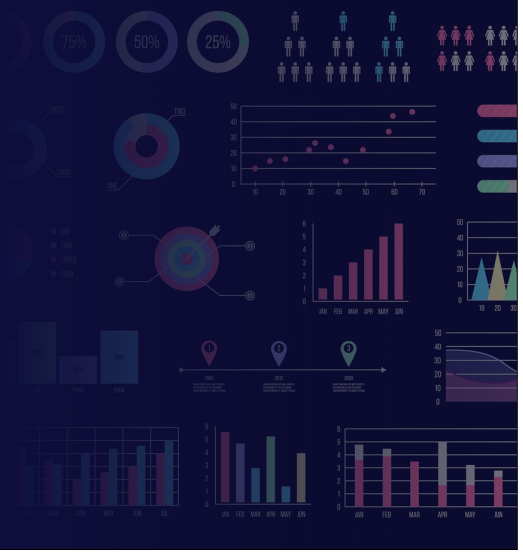

Box Plot
0
20
40
60
80
100
score


Cumulative Histogram
cumulative #evals
0
2
4
6
8
0
20
40
60
80
100
score

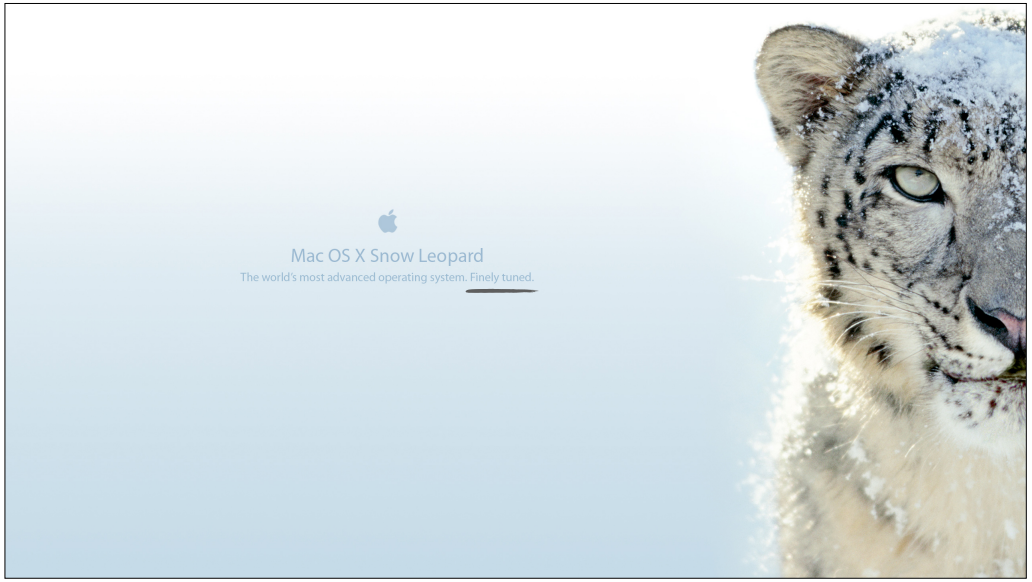

Box Plot
0
20
40
60
80
100
score

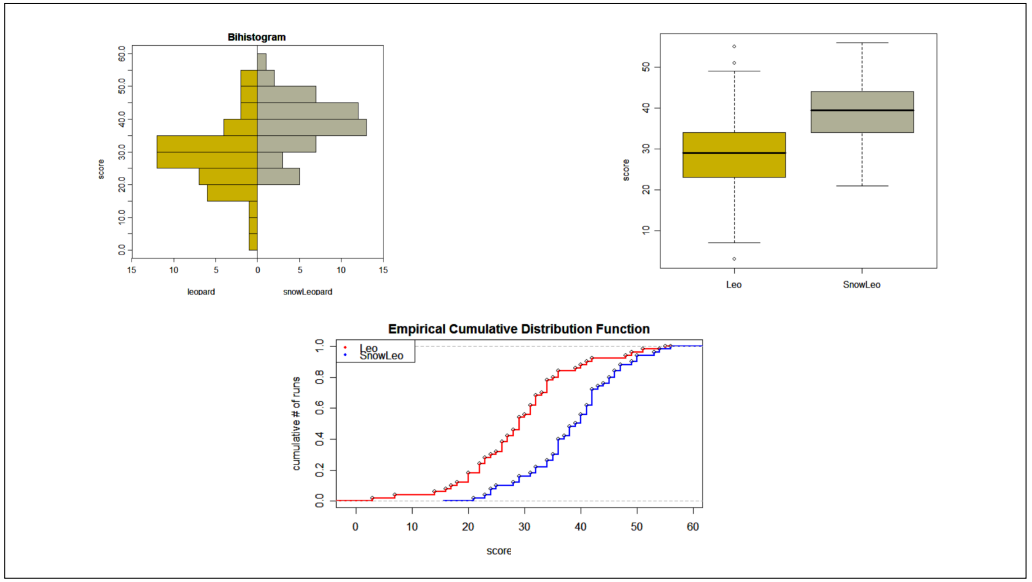

Empirical Cumulative Distribution Function
cumulative #evals
0.0
0.4
0.8
0
20
40
60
80
100
score

Visually comparing distributions

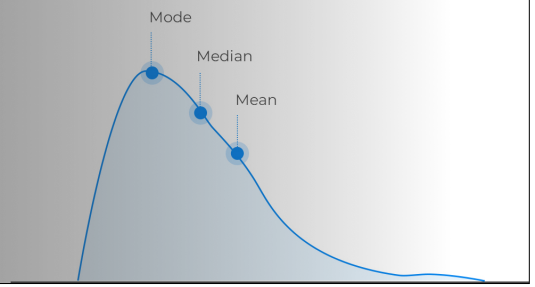








Measures of Central Tendency

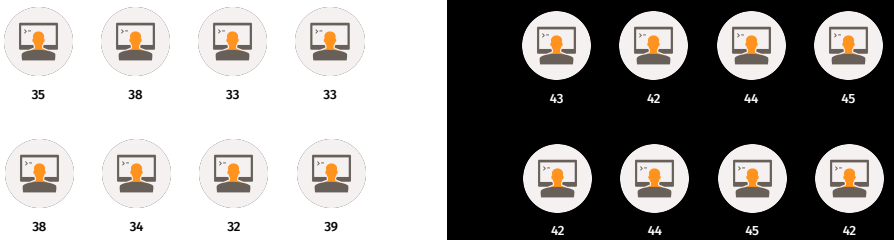


Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*

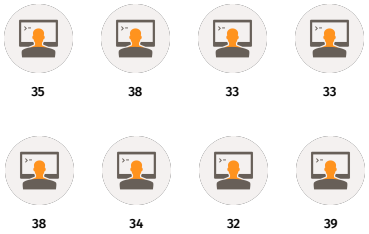
Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*



Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*



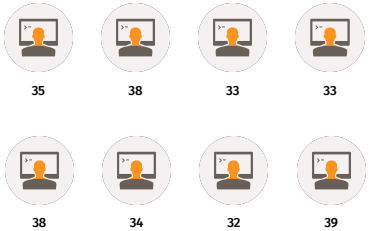
Mean = 35.3



Mean = 43.4

Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*



Mean = 35.3

Median = 34.5



Mean = 43.4

Median = 43.5

Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*



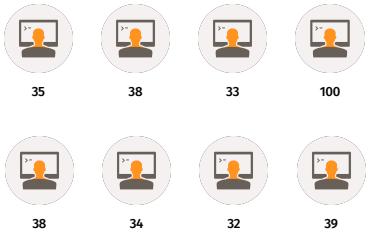
Mean = 43.6



Mean = 43.4

Mean *vs* Median

*I compare the **productivity** of developers when using two code recommender systems.*



Mean = 43.6

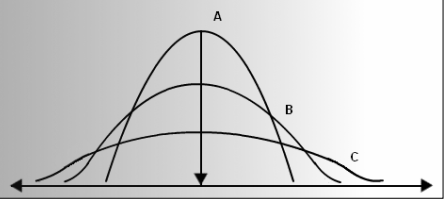
Median = 36.5



Mean = 43.4

Median = 43.5

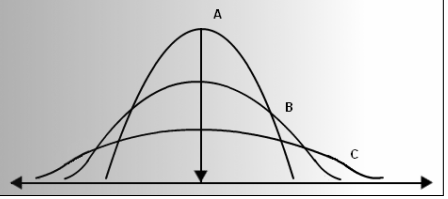
Measures of Dispersion



Measures of Dispersion

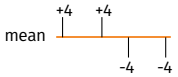
Range (max-min)

IQR



Absolute deviation *vs* Standard deviation

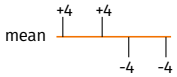
Difference from the mean



$$\frac{4 + 4 - 4 - 4}{4} = 0$$

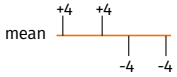
Absolute deviation *vs* Standard deviation

Difference from the mean



$$\frac{4 + 4 - 4 - 4}{4} = 0$$

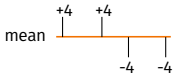
Mean absolute deviation from the mean



$$\frac{|4| + |4| + |-4| + |-4|}{4} = 4$$

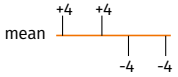
Absolute deviation *vs* Standard deviation

Difference from the mean

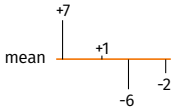


$$\frac{4 + 4 - 4 - 4}{4} = 0$$

Mean absolute deviation from the mean



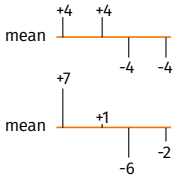
$$\frac{|4| + |4| + |-4| + |-4|}{4} = 4$$



$$\frac{|7| + |1| + |-6| + |-2|}{4} = 4$$

Absolute deviation *vs* Standard deviation

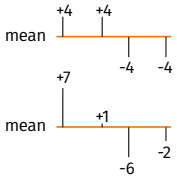
Mean absolute deviation from the mean



$$\frac{|4| + |4| + |-4| + |-4|}{4} = 4$$

$$\frac{|7| + |1| + |-6| + |-2|}{4} = 4$$

Standard deviation



$$\sqrt{\frac{4^2 + 4^2 + (-4)^2 + (-4)^2}{4}} = 4$$

$$\sqrt{\frac{7^2 + 1^2 + (-6)^2 + (-2)^2}{4}} = 4.74$$

Experiment simulation

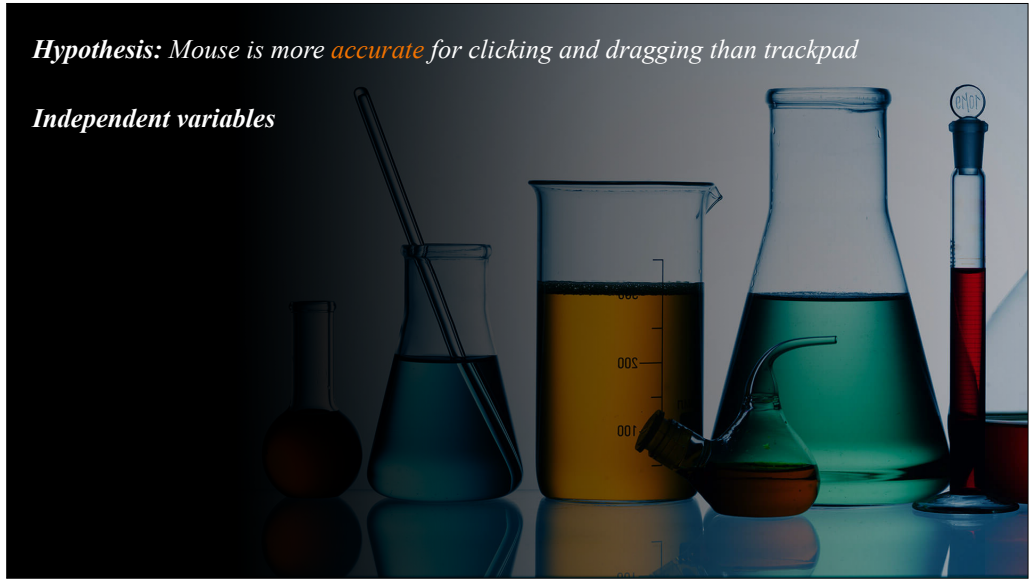


Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*



Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*

Independent variables



Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*

Independent variables

Device	Mouse	Trackpad



Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*

Independent variables

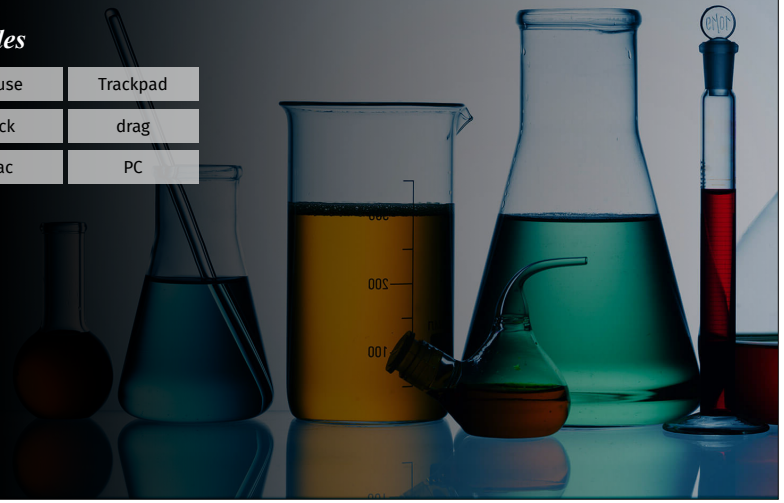
Device	Mouse	Trackpad
Task	click	drag



Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*

Independent variables

Device	Mouse	Trackpad
Task	click	drag
Platform	Mac	PC



Hypothesis: *Mouse is more accurate for clicking and dragging than trackpad*

Independent variables

Device	Mouse	Trackpad
Task	click	drag
Platform	Mac	PC

Dependent variables

Accuracy	Ratio scale (0% ... 100%)
Duration	Ratio scale (ms)





Excel
Basic feature, tables, easy to use.
Visualization: predefined templates.



R
Advanced, cool packages, data frames, language to learn.
Visualization: ggplot2.



Python pandas
Advanced, data frames, using Python.
Visualization: inspired by ggplot2, e.g., seaborn.





Device	Task	Platform	Participant	DateTime	Accuracy	Duration
mouse	click	Mac	P1	2020-01-01 16:23:09	98	120
trackpad	click	Mac	P1	2020-01-01 16:28:19	95	113
mouse	drag	Mac	P1	2020-01-02 17:20:03	67	134
trackpad	drag	Mac	P1	2020-01-02 17:27:19	88	109
mouse	click	PC	P2	2020-01-09 16:13:02	15	154
trackpad	click	PC	P2	2020-01-10 16:28:45	27	187

1 row per observation

Import the Pandas library

```
[1]  1  import pandas as pd
```

Generate Synthetic Data

We normally would just import data from a spreadsheet, but here we generate artificail data to have something interesting to look at.

```
[3]  1  devices = ["mouse", "trackpad"]
     2  tasks = ["click", "drag"]
     3  platforms = ["mac", "pc"]
     4  repetitions = range(4) #This creates a sequence of integers 0, 1, 2, 3
     5  participants = list(map(lambda x: "P"+str(x), range(5)))

[4]  1  data = [{'Device': device, 'Task': task, 'Platform': platform, 'Participant': participant, 'Repetition': repetition} for device in devices for

[7]  1  from datetime import datetime
     2  from datetime import timedelta
     3  dateTimes = list(map((lambda x: datetime.now() + timedelta(minutes=x)), range(len(data))))

[8]  1  import random
     2  import numpy
     3  accuracies = [numpy.clip(random.gauss(40 if data[x]['Device']=='trackpad' else 80, 10), 0, 100) for x in range(len(data))]
     4  durations = [max(0, random.gauss(120 if data[x]['Device']=='trackpad' else 150, 30)) for x in range(len(data))]

[10] 1  frame = pd.DataFrame(data)
     2  frame['StartTime'] = dateTimes
     3  frame['Accuracy'] = accuracies
     4  frame['Duration'] = durations

[11] 1  frame.to_csv('data.csv', index=False)
```

https://colab.research.google.com/drive/1m9zwwYspKaJqa_05iqBf51JPnHcU_E77?usp=sharing

Data Wrangling with pandas Cheat Sheet

http://pandas.pydata.org

Tidy Data – A foundation for wrangling in pandas

In a tidy data set: T M A & T M A

Each **variable** is saved in its own **column**

Each **observation** is saved in its own **row**

Tidy data complements pandas's vectorized operations. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

M * A

Syntax – Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

		a	b	c
x	v			
d	1	4	7	10
	2	5	8	11
e	1	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
        [('d',1),('d',2),('e',2)],
         names=['n','v']))
```

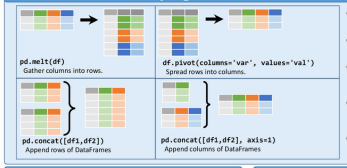
Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
        .rename(columns={
                'variable' : 'var',
                'value' : 'val'})
        .query('val >= 200')
    )
```

Reshaping Data – Change the layout of a data set



`pd.melt(df)`
Gather columns into rows.

`df.pivot(columns='var', values='val')`
Spread rows into columns.

`pd.concat([df1,df2])`
Append rows of DataFrames

`pd.concat([df1,df2], axis=1)`
Append columns of DataFrames

`df=df.assign(a=[1:3], b=[4:6])`
Add new columns to DataFrame.

`df=df.sort_values('mpg')`
Order rows by values of a column (low to high).

`df=df.sort_values('mpg', ascending=False)`
Order rows by values of a column (high to low).

`df=df.rename(columns={'y':'year'})`
Rename the columns of a DataFrame

`df=df.sort_index()`
Sort the index of a DataFrame

Subset Observations (Rows)



`df[df.Length > 7]`
Extract rows that meet logical criteria.

`df.drop_duplicates()`
Remove duplicate rows (only considers columns).

`df.sample(frac=0.5)`
Randomly select fraction of rows.

`df.sample(n=10)`
Randomly select n rows.

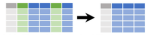
`df.iloc[10:20]`
Select rows by position.

`df.nlargest(10, 'value')`
Select and order top n entries.

Logic in Python (and pandas)

<	Less than	!=	Not equal to
>	Greater than	`df.column.isin(values)`	Group membership
==	Equals	`pd.isnull(obj)`	Is NaN
<=	Less than or equals	`pd.notnull(obj)`	Is not NaN
>=	Greater than or equals	`&,|,~,^,df.any(),df.all()`	Logical and, or, not, xor, any, all

Subset Variables (Columns)



`df[['width','length','species']]`
Select multiple columns with specific names.

`df['width']` or `df.width`
Select single column with specific name.

`df.filter(regex='regex')`
Select columns whose name matches regular expression *regex*.

regex (Regular Expressions) Examples

`'\.'`	Matches strings containing a period '.'
`'Length$'`	Matches strings ending with word 'Length'
`'^Sepal'`	Matches strings beginning with the word 'Sepal'
`'^x[1-5]$'`	Matches strings beginning with 'x' and ending with 1,2,3,4,5
`'^(?!Species$).*'`	Matches strings except the string 'Species'

`df.loc[:,'x2':'x4']`
Select all columns between x2 and x4 (inclusive).

`df.iloc[:,[1,2,5]]`
Select columns in positions 1, 2 and 5 (first column is 0).

`df.loc[df['a'] > 10, ['a','c']]`
Select rows meeting logical condition, and only the specific columns .

Summarize Data

`df['length'].value_counts()`
Count number of rows with each unique value of variable

`len(df)`
of rows in DataFrame.

`len(df['w'].unique())`
of distinct values in a column.

`df.describe()`
Basic descriptive statistics for each column (or GroupBy)



pandas provides a large set of **summary functions** that operate on different kinds of pandas objects (DataFrame columns, Series, GroupBy, Expanding and Rolling (see below)) and produce single values for each of the groups. When applied to a DataFrame, the result is returned as a pandas Series for each column. Examples:

`sum()`
Sum values of each object.

`count()`
Count non-NA/null values of each object.

`median()`
Median value of each object.

`quantile([0.25,0.75])`
Quantiles of each object.

`apply(function)`
Apply function to each object.

`min()`
Minimum value in each object.

`max()`
Maximum value in each object.

`mean()`
Mean value of each object.

`var()`
Variance of each object.

`std()`
Standard deviation of each object.

Group Data



`df.groupby(by="col")`
Return a GroupBy object, grouped by values in column named "col".

`df.groupby(level="ind")`
Return a GroupBy object, grouped by values in index level named "ind".

All of the summary functions listed above can be applied to a group. Additional GroupBy functions:

`size()`
Size of each group.

`agg(function)`
Aggregate group using function.

The examples below can also be applied to groups. In this case, the function is applied on a per-group basis, and the returned vectors are of the length of the original DataFrame.

`shift(1)`
Copy with values shifted by 1.

`rank(method='dense')`
Ranks with no gaps.

`rank(method='min')`
Ranks. Ties get min rank.

`rank(pct=True)`
Ranks rescaled to interval [0, 1].

`rank(method='first')`
Ranks. Ties go to first value.

`shift(-1)`
Copy with values lagged by 1.

`cumsum()`
Cumulative sum.

`cummax()`
Cumulative max.

`cummin()`
Cumulative min.

`cumprod()`
Cumulative product.

Windows

`df.expanding()`
Return an Expanding object allowing summary functions to be applied cumulatively.

`df.rolling(n)`
Return a Rolling object allowing summary functions to be applied to windows of length n.

Handling Missing Data

`df=df.dropna()`
Drop rows with any column having NA/null data.

`df=df.fillna(value)`
Replace all NA/null data with value.

Make New Variables

`df=df.assign(Area=lambda df: df.Length*df.Height)`
Compute and append one or more new columns.

`df['Volume'] = df.Length*df.Height*df.Depth`
Add single column.

`pd.qcut(df.col, n, labels=False)`
Bin column into n buckets.



pandas provides a large set of **vector functions** that operate on all columns of a DataFrame or a single selected column (a pandas Series). These functions produce vectors of values for each of the columns, or a single Series for the individual Series. Examples:

`max(axis=1)`
Element-wise max.

`min(axis=1)`
Element-wise min.

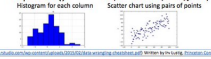
`clip(lower=-10,upper=10)`
Trim values at input thresholds

`abs()`
Absolute value.

Plotting

`df.plot.hist()`
Histogram for each column

`df.plot.scatter(x='w',y='h')`
Scatter chart using pairs of points



Combine Data Sets

adf

x1	x2
A	1
B	2
C	3

bdf

x1	x3
A	T
B	F
D	T

Standard Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NaN

```
pd.merge(adf, bdf,
         how='left', on='x1')
```

Join matching rows from bdf to adf.

x1	x2	x3
A	1.0	T
B	2.0	F
D	NaN	T

```
pd.merge(adf, bdf,
         how='right', on='x1')
```

Join matching rows from adf to bdf.

x1	x2	x3
A	1	T
B	2	F

```
pd.merge(adf, bdf,
         how='inner', on='x1')
```

Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NaN
D	NaN	T

```
pd.merge(adf, bdf,
         how='outer', on='x1')
```

Join data. Retain all values, all rows.

Filtering Joins

x1	x2
A	1
B	2

```
adf[adf.x1.isin(bdf.x1)]
```

All rows in adf that have a match in bdf

x1	x2
C	3

```
adf[~adf.x1.isin(bdf.x1)]
```

All rows in adf that do not have a match in bdf.

ydf

x1	x2
A	1
B	2
C	3

zdf

x1	x2
B	2
C	3
D	4

Set-like Operations

x1	x2
B	2
C	3

```
pd.merge(ydf, zdf)
```

Rows that appear in both ydf and zdf (Intersection).

x1	x2
A	1
B	2
C	3
D	4

```
pd.merge(ydf, zdf, how='outer')
```

Rows that appear in either or both ydf and zdf (Union).

x1	x2
A	1

```
pd.merge(ydf, zdf, how='outer',
         indicator=True)
.query('_merge == "left_only"')
.drop(['_merge'],axis=1)
```

Rows that appear in ydf but not zdf (Setdiff).